SIMATS ENGINEERING

ASSESSMENT TOOL 1 -Constraint Based Problem Solving

COURSE NAME: R PROGRAMMING **COURSE CODE: ITA0402**

COURSE OUTCOME COVERED

CO1: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively. Identify and implement appropriate control structures to solve a particular programming problem
(Bloom Level -BL3- BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5 Questions **Total Marks: 100**

Code of Conduct:

*I **A. Midhuna (192521302)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

A. Midhuna
Signature of the student:

1. Student Performance Analysis

A college stores the marks of 10 students in Mathematics, Physics, and Chemistry. Write an R program to:

- Store the marks in a matrix.
- Calculate subject-wise averages.
- Find the highest and lowest scorer in each subject.
- Display students scoring above the overall average.

SOLN:

```
# Student Performance Analysis

# Store marks in a matrix

marks <- matrix (c(

  85, 78, 90,

  72, 88, 76,

  91, 95, 89,

  65, 70, 68,

  80, 82, 85,

  75, 69, 74,

  88, 91, 93,

  60, 64, 59,

  95, 89, 97,

  78, 85, 80

), nrow = 10, byrow = TRUE)

colnames(marks) <- c("Mathematics", "Physics", "Chemistry")

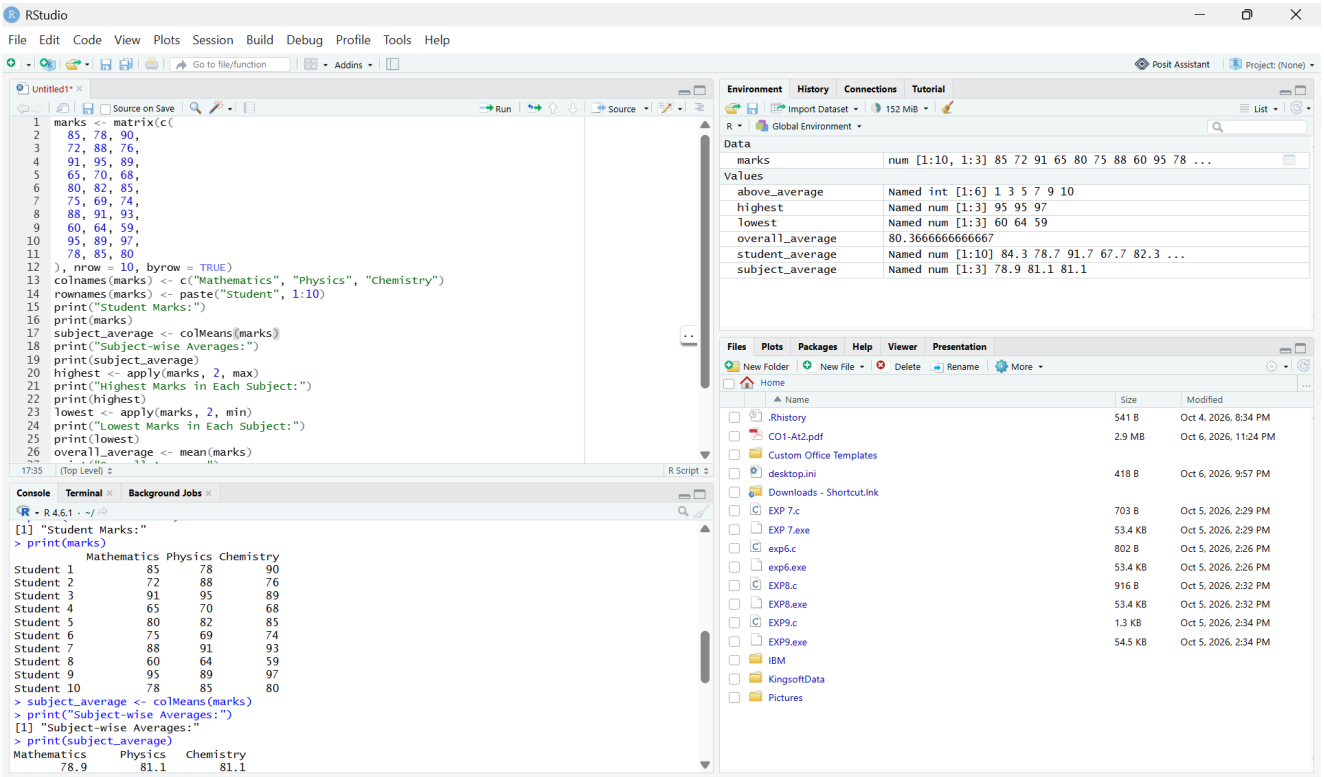
rownames(marks) <- paste("Student", 1:10)

print("Student Marks:")

print(marks)

# Subject-wise averages
```

```
subject_average <- colMeans(marks)
print("Subject-wise Averages:")
print(subject_average)
# Highest scorer in each subject
highest <- apply(marks, 2, max)
print("Highest Marks in Each Subject:")
print(highest)
# Lowest scorer in each subject
lowest <- apply(marks, 2, min)
print("Lowest Marks in Each Subject:")
print(lowest)
# Overall average
overall_average <- mean(marks)
print("Overall Average:")
print(overall_average)
# Students scoring above overall average
student_average <- rowMeans(marks)
above_average <- which(student_average > overall_average)
print("Students Scoring Above Overall Average:")
print(above_average)
```



2. Employee Salary Processing

An organization maintains employee details including Employee ID, Name, Basic Pay, and Allowance.
Write an R program using appropriate data structures to:

- Calculate Gross Salary.
- Identify employees earning above ₹50,000.
- Display the employee with the highest salary.
- Generate a summary report.

SOLN:

Employee Salary Processing

Create employee data frame

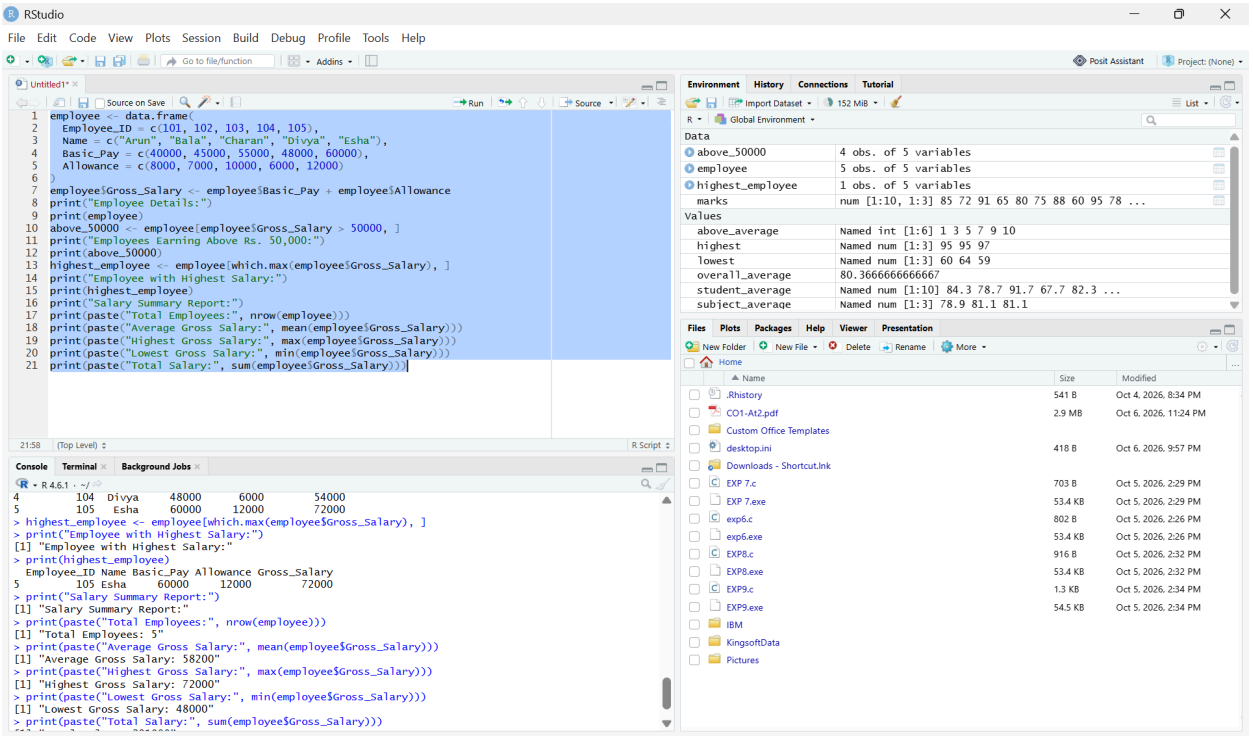
employee <- data.frame(

Employee_ID = c(101, 102, 103, 104, 105),

Name = c("Arun", "Bala", "Charan", "Divya", "Esha"),

Basic_Pay = c(40000, 45000, 55000, 48000, 60000),

```
  Allowance = c(8000, 7000, 10000, 6000, 12000)
)
# Calculate Gross Salary
employee$Gross_Salary <- employee$Basic_Pay + employee$Allowance
print("Employee Details:")
print(employee)
# Employees earning above Rs. 50,000
above_50000 <- employee[employee$Gross_Salary > 50000, ]
print("Employees Earning Above Rs. 50,000:")
print(above_50000)
# Employee with highest salary
highest_employee <- employee[which.max(employee$Gross_Salary), ]
print("Employee with Highest Salary:")
print(highest_employee)
# Summary report
print("Salary Summary Report:")
print(paste("Total Employees:", nrow(employee)))
print(paste("Average Gross Salary:", mean(employee$Gross_Salary)))
print(paste("Highest Gross Salary:", max(employee$Gross_Salary)))
print(paste("Lowest Gross Salary:", min(employee$Gross_Salary)))
print(paste("Total Salary:", sum(employee$Gross_Salary)))
```



3. Weather Data Analysis Using Arrays

Daily temperature data for three cities is collected for 7 days.
Write an R program to:

- Store the data in a 3-dimensional array.
- Find the average temperature for each city.
- Identify the hottest day.
- Compare city-wise temperature variations.

SOLN:

```
# Weather Data Analysis Using Arrays

# Temperature data for 3 cities and 7 days

temperature <- array(c(

  30, 32, 31, 33, 35, 34, 36,

  28, 29, 30, 31, 32, 33, 34,

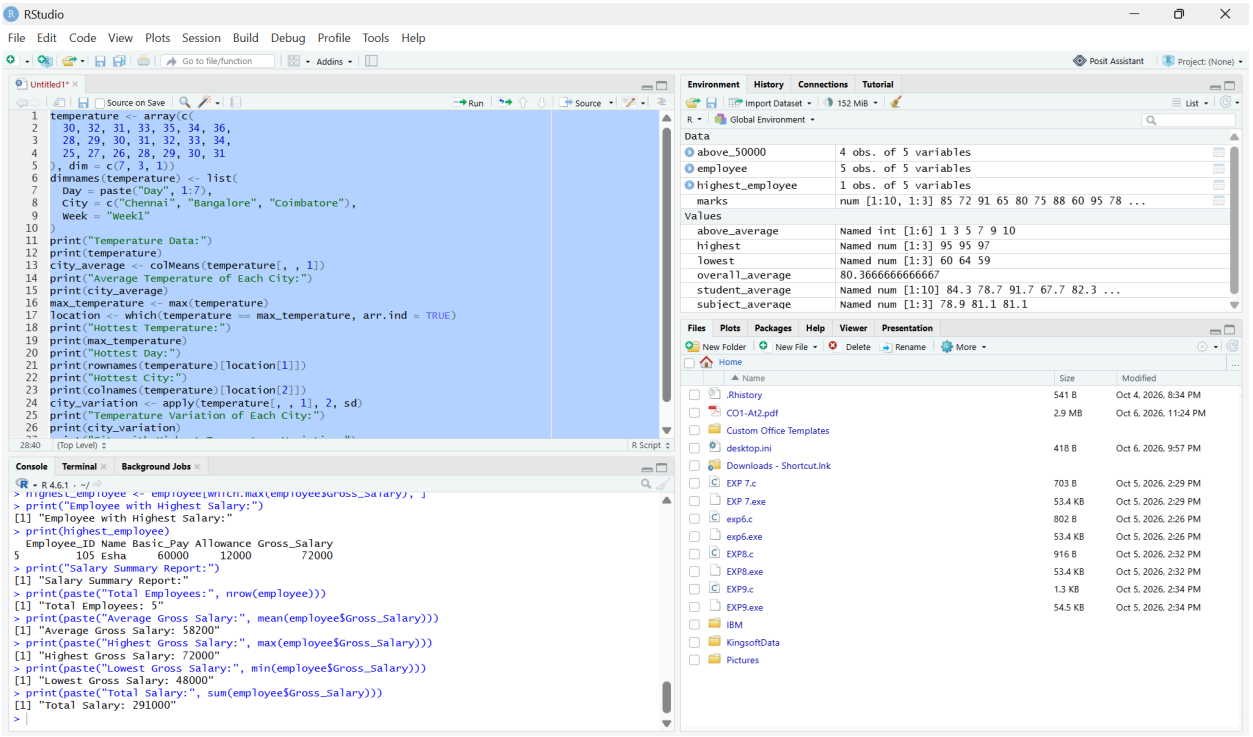
  25, 27, 26, 28, 29, 30, 31

), dim = c(7, 3, 1))

dimnames(temperature) <- list(

  Day = paste("Day", 1:7),
```

```
  City = c("Chennai", "Bangalore", "Coimbatore"),
  Week = "Week1"
)
print("Temperature Data:")
print(temperature)
# Average temperature for each city
city_average <- colMeans(temperature[, , 1])
print("Average Temperature of Each City:")
print(city_average)
# Find hottest day
max_temperature <- max(temperature)
location <- which(temperature == max_temperature, arr.ind = TRUE)
print("Hottest Temperature:")
print(max_temperature)
print("Hottest Day:")
print(rownames(temperature)[location[1]])
print("Hottest City:")
print(colnames(temperature)[location[2]])
# Compare city-wise temperature variations
city_variation <- apply(temperature[, , 1], 2, sd)
print("Temperature Variation of Each City:")
print(city_variation)
print("City with Highest Temperature Variation:")
print(names(which.max(city_variation)))
```



4. Product Inventory Management

A store maintains product information such as Product ID, Product Name, Quantity, and Price. Using lists and vectors in R:

- Calculate inventory value for each product.
- Find products with stock below a threshold value.
- Generate a stock summary report.

SOLN:

Product Inventory Management

Product information using vectors

product_id <- c(101, 102, 103, 104, 105)

product_name <- c(

"Laptop",

"Mouse",

"Keyboard",

"Monitor",

"Printer"

```
)
quantity <- c(10, 50, 30, 15, 8)
price <- c(50000, 800, 1500, 12000, 15000)
# Create a list
inventory <- list(
  Product_ID = product_id,
  Product_Name = product_name,
  Quantity = quantity,
  Price = price
)
print("Product Inventory:")
print(inventory)
# Calculate inventory value
inventory_value <- quantity * price
# Add inventory value to the list
inventory$Inventory_Value <- inventory_value
print("Inventory Value of Each Product:")
print(inventory_value)
# Threshold value
threshold <- 10
# Products with stock below threshold
low_stock <- inventory$Product_Name[inventory$Quantity < threshold]
print("Products with Stock Below Threshold:")
print(low_stock)
```

Stock summary report

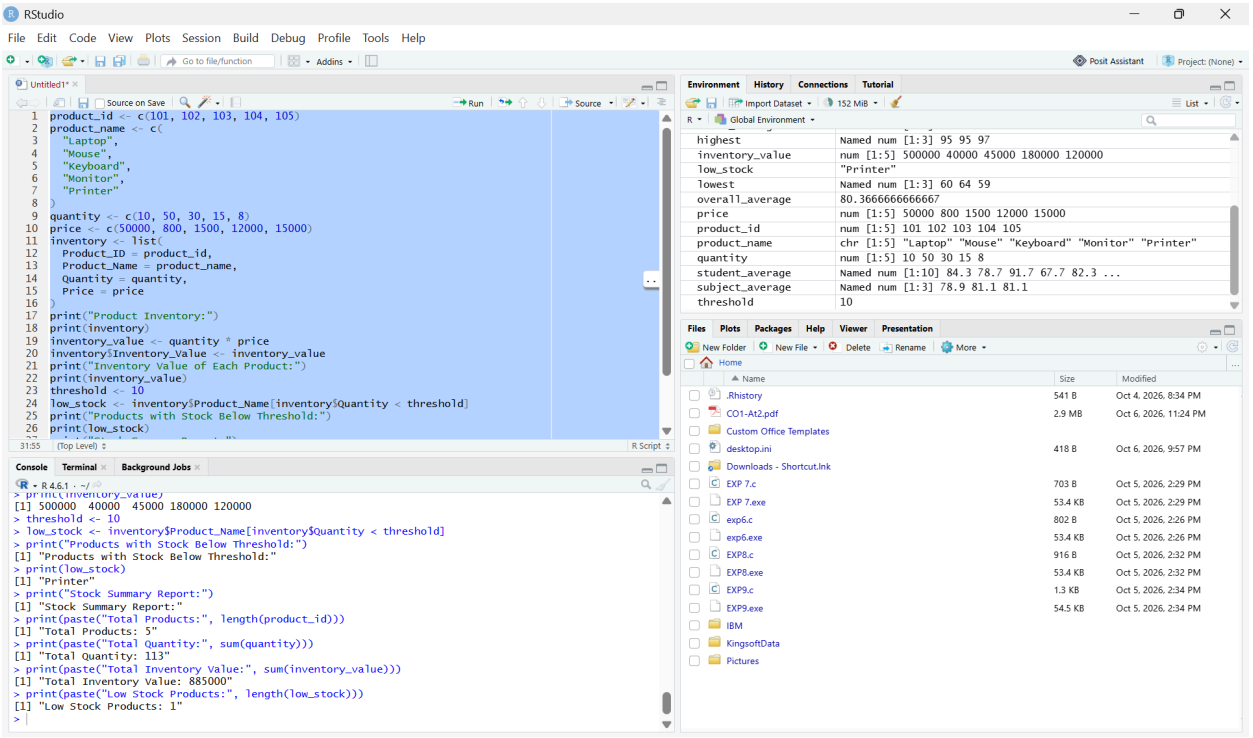
print("Stock Summary Report:")

print(paste("Total Products:", length(product_id)))

print(paste("Total Quantity:", sum(quantity)))

print(paste("Total Inventory Value:", sum(inventory_value)))

print(paste("Low Stock Products:", length(low_stock)))



5. Bank Account Processing Using Functions

Develop an R program with user-defined functions to:

- Deposit money.
- Withdraw money.
- Check account balance.
- Display transaction history.
  Use suitable variables and functions.

SOLN:

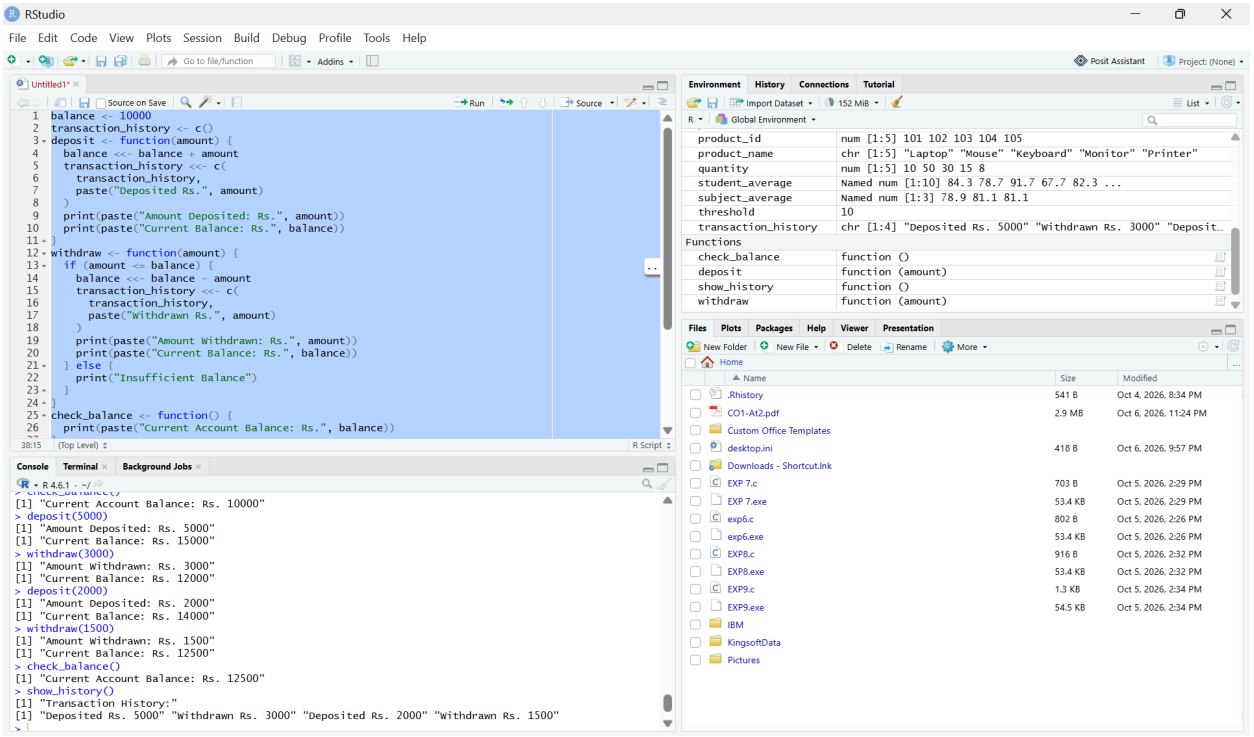
Bank Account Processing Using Functions

Initial balance

balance <- 10000

```
# Transaction history
transaction_history <- c()
# Function for deposit
deposit <- function(amount) {
  balance <<- balance + amount
  transaction_history <<- c(
    transaction_history,
    paste("Deposited Rs.", amount)
  )
  print(paste("Amount Deposited: Rs.", amount))
  print(paste("Current Balance: Rs.", balance))
}
# Function for withdrawal
withdraw <- function(amount) {
  if (amount <= balance) {
    balance <<- balance - amount
    transaction_history <<- c(
      transaction_history,
      paste("Withdrawn Rs.", amount)
    )
    print(paste("Amount Withdrawn: Rs.", amount))
    print(paste("Current Balance: Rs.", balance))
  } else {
    print("Insufficient Balance")
```

```
  }
}
# Function to check balance
check_balance <- function() {
  print(paste("Current Account Balance: Rs.", balance))
}
# Function to display transaction history
show_history <- function() {
  print("Transaction History:")
  print(transaction_history)
}
# Perform transactions
check_balance()
deposit(5000)
withdraw(3000)
deposit(2000)
withdraw(1500)
# Display final balance
check_balance()
# Display transaction history
show_history()
```


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Untitled1*
balance <- 10000
transaction_history <- c()
deposit <- function(amount) {
balance <<- balance + amount
transaction_history <<- c(
transaction_history,
paste("Deposited Rs.", amount)
)
print(paste("Amount Deposited: Rs.", amount))
print(paste("Current Balance: Rs.", balance))
}
withdraw <- function(amount) {
if (amount <= balance) {
balance <<- balance - amount
transaction_history <<- c(
transaction_history,
paste("Withdrawn Rs.", amount)
)
print(paste("Amount Withdrawn: Rs.", amount))
print(paste("Current Balance: Rs.", balance))
} else {
print("Insufficient Balance")
}
}
check_balance <- function() {
print(paste("Current Account Balance: Rs.", balance))
}
Console Terminal Background Jobs
[1] "Current Account Balance: Rs. 10000"
> deposit(5000)
[1] "Amount Deposited: Rs. 5000"
[1] "Current Balance: Rs. 15000"
> withdraw(3000)
[1] "Amount Withdrawn: Rs. 3000"
[1] "Current Balance: Rs. 12000"
> deposit(2000)
[1] "Amount Deposited: Rs. 2000"
[1] "Current Balance: Rs. 14000"
> withdraw(1500)
[1] "Amount Withdrawn: Rs. 1500"
[1] "Current Balance: Rs. 12500"
> check_balance()
[1] "Current Account Balance: Rs. 12500"
> show_history()
[1] "Transaction History:"
[1] "Deposited Rs. 5000" "Withdrawn Rs. 3000" "Deposited Rs. 2000" "Withdrawn Rs. 1500"
Environment History Connections Tutorial
product_id num [1:5] 101 102 103 104 105
product_name chr [1:5] "Laptop" "Mouse" "Keyboard" "Monitor" "Printer"
quantity num [1:5] 10 50 30 15 8
student_average Named num [1:10] 84.3 78.7 91.7 67.7 82.3 ...
subject_average Named num [1:3] 78.9 81.1 81.1
threshold 10
transaction_history chr [1:4] "Deposited Rs. 5000" "Withdrawn Rs. 3000" "Deposit...
Functions
check_balance function ()
deposit function (amount)
show_history function ()
withdraw function (amount)
Files Plots Packages Help Viewer Presentation
Name Size Modified
.Rhistory 541 B Oct 4, 2026, 8:34 PM
CO1-At2.pdf 2.9 MB Oct 6, 2026, 11:24 PM
Custom Office Templates
desktop.ini 418 B Oct 6, 2026, 9:57 PM
Downloads - Shortcut.lnk
EXP 7.c 703 B Oct 5, 2026, 2:29 PM
EXP 7.exe 53.4 KB Oct 5, 2026, 2:29 PM
exp6.c 802 B Oct 5, 2026, 2:26 PM
exp6.exe 53.4 KB Oct 5, 2026, 2:26 PM
EXP8.c 916 B Oct 5, 2026, 2:32 PM
EXP8.exe 53.4 KB Oct 5, 2026, 2:32 PM
EXP9.c 1.3 KB Oct 5, 2026, 2:34 PM
EXP9.exe 54.5 KB Oct 5, 2026, 2:34 PM
IBM
KingsoftData
Pictures

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand